

Temporal CNN for DDoS Attack Detection

Lokesh L K S (lj15837@psu.edu)

The Pennsylvania State University

8th February 2026

1. Research Methodology

1.1 Data Acquisition and Preprocessing

Status: Completed in Weeks 1-2

I'm using the BCCC-cPacket-Cloud-DDoS-2024 dataset from UNB's Canadian Institute for Cybersecurity. The dataset has 17 different DDoS attack scenarios with 300+ network flow features. I've already downloaded it and completed the initial data exploration.

Completed Preprocessing Steps:

- Downloaded BCCC-cPacket-Cloud-DDoS-2024 dataset from UNB CIC
- Extracted network flow statistics and explored feature distributions
- Implemented basic data cleaning and normalization to [0-255] range
- Created initial train/validation/test splits (70%/15%/15%) with stratified sampling
- Applied SMOTE for class balancing between attack and benign samples

Current Work (Week 4):

- Finalizing temporal image conversion pipeline - grouping consecutive flows into time windows
- Testing different time window sizes (5s, 10s, 20s, 50s) to see impact on pattern capture
- Implementing efficient batch processing for temporal image generation

1.2 AI/ML Model Selection and Architecture

Baseline CNN Status: Initial implementation completed in Week 3

I chose CNNs because they're really good at learning spatial patterns from image-like data. Since my approach converts sequential network flows into 2D temporal images, CNNs can learn both spatial relationships between flow features and temporal patterns of how attacks evolve. I've already built and tested a baseline CNN, and now I'm working on the temporal image pipeline to feed it properly.

Baseline CNN Architecture (Already Implemented):

- Input Layer: Currently testing different input shapes for temporal images
- Conv2D Layer 1: 32 filters, 3×3 kernel, ReLU activation
- MaxPooling2D: 2×2 pooling
- Conv2D Layer 2: 64 filters, 3×3 kernel, ReLU activation
- MaxPooling2D: 2×2 pooling
- Flatten Layer: Convert to 1D for classification
- Dense Layer: 128 neurons, ReLU activation
- Dropout: 0.5 to prevent overfitting

- Output Layer: 1 neuron, sigmoid activation (binary classification)

Training Configuration:

- Loss Function: Binary cross-entropy
- Optimizer: Adam with learning rate 0.001
- Batch Size: 64 (testing 32, 64, 128)
- Epochs: Up to 50 with early stopping (patience=10)
- Metrics: Accuracy, Precision, Recall, F1-Score, confusion matrix.
- Framework: TensorFlow 2.x with Keras API

Planned Optimizations (Weeks 8-11):

- Reduce model complexity.
- Test shallower architectures with fewer filters
- Experiment with 1D convolutions instead of 2D to reduce parameters
- Hyperparameter tuning: learning rate, batch size, dropout rate

Optional LSTM Comparison (Week 14): If time permits after achieving 80% accuracy with CNN, I'll implement an LSTM baseline to compare how well recurrent architectures handle temporal sequences vs. the temporal image CNN approach. This is a stretch goal, not critical path.

1.3 Cybersecurity Risk Assessment

Since this could potentially be deployed in production environments for DDoS detection, I need to consider security risks with the model itself and deployment scenarios.

- **Adversarial Attacks:** Attackers could craft flows to evade detection. *Mitigation:* Test robustness with adversarial examples during evaluation; implement input validation
- **False Negatives:** Missing real attacks leaves systems vulnerable. *Mitigation:* Tune threshold to favor recall; prioritize minimizing FNR over FPR; set alert system
- **Resource Exhaustion:** Model could be DoS target through excessive inference requests. *Mitigation:* Keep model lightweight ;
- **Data Privacy:** Network flow data might contain sensitive info. *Mitigation:* Use only flow statistics, not payloads; anonymize identifying information

2. System Architecture

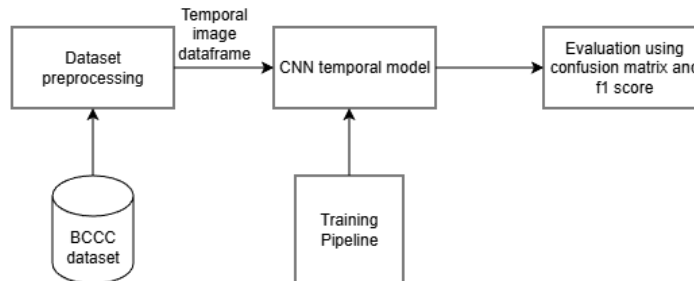
The system has four main components: Data Preprocessing Pipeline (mostly done), Temporal CNN Model (baseline complete, optimization pending), Evaluation Framework (in progress). Everything is modular so I can test and improve components independently.

System Components and Status:

- **Data Preprocessing Pipeline:** Complete - converts flows to temporal images, SMOTE balancing, train/val/test splits
- **Temporal CNN Model:** In Progress - baseline working, temporal image integration underway, optimization pending
- **Evaluation Framework:** In Progress - basic metrics implemented and resource profiling planned.

System Architecture Diagram

- Data Flow: BCCC dataset → preprocessing → temporal images (COMPLETED sections marked)
- CNN Model: Input layer → Conv layers → Pooling → Dense → Output (with parameter counts)
 - Training Pipeline: Training loop, validation, early stopping
 - Evaluation Pipeline: Test metrics, ablation studies, resource profiling



3. Project Plan and Timeline (15 Weeks)

The project runs 15 weeks from January to May 2026, with hard deadline April 30. I'm currently in Week 4 finishing the temporal image pipeline. The timeline is realistic with buffer time for issues.

3.1 Progress Update (Week 4)

Completed Work (Weeks 1-4):

- Week 1-2: Downloaded BCCC dataset, completed data exploration and cleaning
- Week 3: Implemented baseline CNN architecture, initial training runs
- Week 4 (current): Finalizing temporal image conversion pipeline, testing time window sizes

3.2 Detailed Timeline and Milestones

Phase 1: Foundation (Weeks 1-4) -MOSTLY COMPLETE

- **Week 1-2:** Dataset acquisition, exploration, initial preprocessing [Done]
- **Week 3:** Baseline CNN implementation and initial training [\ Done]
- **Week 4:** Temporal image pipeline development and testing [In Progress]

Deliverable: Deliverable: Dataset ready, baseline CNN working, temporal pipeline operational

Phase 2: Baseline Benchmarking (Weeks 5-7)

- **Week 5:** Complete temporal CNN integration, run initial benchmarks [Planned]
- **Week 6:** Baseline performance evaluation, identify bottlenecks [Planned]
- **Week 7:** Resource profiling: memory usage, inference time, parameter counts [Planned]

Deliverable: Deliverable: Baseline metrics established, performance bottlenecks identified

Phase 3: Temporal CNN Optimization (Weeks 8-11)

- **Week 8-9:** Model architecture optimization to reduce parameters [Planned]
- **Week 10:** Hyperparameter tuning: learning rate, batch size, architecture depth [Planned]

- **Week 11:** study: temporal images vs. per-window classification [Planned]

Deliverable: Deliverable: Optimized temporal CNN achieving $\geq 80\%$ accuracy with $< 5,000$ parameters

Phase 4: Evaluation and Optional Extensions (Weeks 12-14)

- **Week 12:** Comprehensive evaluation on test set, analyze false positives/negatives [Planned]
- **Week 13:** Test on held-out attack examples, measure inference performance [Planned]
- **Week 14:** Optional: LSTM baseline comparison if time permits [Optional]

Deliverable: Deliverable: Complete evaluation results, performance analysis

Phase 5: Documentation (Week 15)

- **Week 15:** Final report writing, presentation preparation, code documentation [Planned]

Deliverable: Deliverable: All project deliverables ready for submission (deadline: April 30)

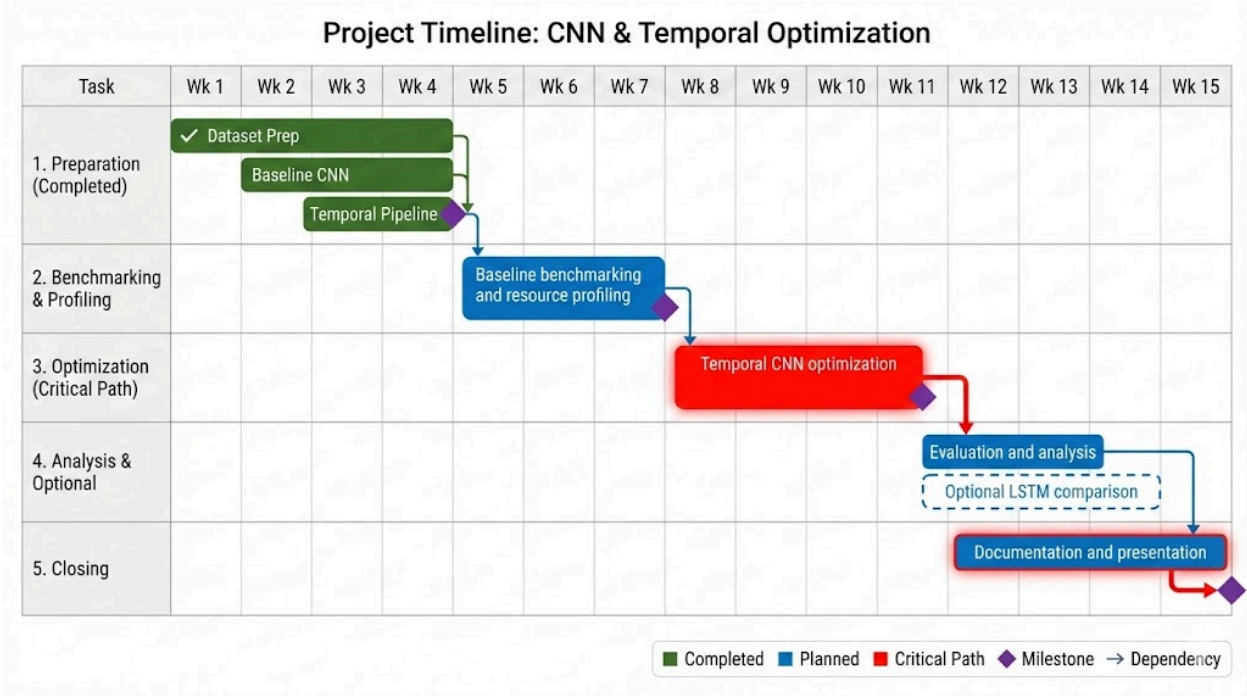
3.3 Critical Path and Dependencies

The critical path runs through: Dataset Prep (Done) → Temporal Pipeline (Week 4) → Baseline Benchmarking (Weeks 5-7) → Optimization (Weeks 8-11) → Evaluation (Weeks 12-13) → Documentation (Week 15). Week 14 LSTM comparison is optional and off critical path.

Key Dependencies:

- Baseline Benchmarking (Weeks 5-7) depends on temporal image pipeline completion (Week 4)
- Optimization (Weeks 8-11) requires baseline metrics from Weeks 5-7
- Final Evaluation (Weeks 12-13) needs optimized model from Weeks 8-11
- Documentation (Week 15) needs all results from evaluation phase
- If temporal pipeline takes extra time in Week 4, Weeks 5-7 can compress slightly without impacting final deadline

3.4 Gantt Chart (15-Week Timeline)



5. Risk Management and Contingency Plans

Based on progress so far and remaining work, here are the main risks and my plans to handle them:

Temporal Pipeline Complexity: Week 4 temporal image conversion taking longer than expected.

Contingency: Simplify image generation if needed; test with just one time window size first; extend into Week 5 if necessary (buffer available)

Can't Hit 80% Accuracy: Model doesn't reach 80% threshold after optimization. *Contingency:* Try different time window sizes; adjust SMOTE ratio; test simpler architectures; focus on getting best possible accuracy rather than parameter optimization

Timeline Slippage: Falling behind due to complexity or debugging. *Contingency:* Skip optional LSTM comparison (Week 14); compress benchmarking phase (Weeks 5-7); prioritize core deliverables

Computational Bottlenecks: Training takes too long or runs out of memory. *Contingency:* Use Google Colab Pro; reduce batch size; train on data subset first; request Penn State cluster access

7. Conclusion and Next Steps

I'm on track with the project timeline. Weeks 1-3 went well - dataset is ready and baseline CNN is working. I'm currently in Week 4 finalizing the temporal image pipeline, which is the last piece needed before I can start serious benchmarking and optimization. The 15-week plan gives me enough buffer time to handle any issues in the optimization phase (Weeks 8-11), which is the most uncertain part. The April 30 deadline is achievable, and I've identified clear contingency plans if things take longer than expected. The modular architecture means I can iterate quickly on the CNN optimization without redoing data preprocessing. Next immediate steps: finish temporal image pipeline this week, then move into baseline benchmarking in Week 5 to establish performance metrics before optimization begins.